

Data 498D

Git and Github

Taryn Laird

Department of Mathematics



Goals for Today

- What is Github?
- Create a Github Account
- Download Github Desktop to use
- Fork a repository and create branches to work on changes independently
- Make contributions to a collaborative project using pull requests
- Conduct code reviews by providing constructive feedback on others' work
- Understand how to identify and resolve merge conflicts when multiple people edit the same files

What is Github?

- GitHub is:
 - a cloud-based platform for version control and collaboration, built on Git.
 - Think of it as a combination of
 - Dropbox for code
 - A time machine for your projects.
- Why it matters
 - For a data science project, you're probably juggling Python notebooks, datasets, model files, documentation, and visualizations. GitHub lets you track every change you make—so if you accidentally break your model or want to revisit an earlier approach, you can roll back.

What is Github?

- Key Vocabulary
 - **Repositories** are like project folders that contain all your code, data (if it's small), and documentation.
 - **Commits** are snapshots of your work at different points—think of them as save points.
 - **Branches** let you experiment with new ideas without messing up your working code.
 - **Pull Requests** if you're working with teammates, GitHub makes it easy to merge everyone's contributions without overwriting each other's work.

Git and Github Activity – Getting Started

Create a Github Account

- Go to <https://github.com/>
- Sign in if you already have an account
- Sign-up if you do not already have an account



Platform ▾ Solutions ▾ Resources ▾ Open Source ▾ Enterprise ▾ Pricing

Search or jump to...

Sign in

Sign up



Git and Github Activity – Getting Started

Download Github Desktop

- Go to <https://desktop.github.com/download/>
- Download the appropriate version for your device
- Once you have downloaded, open the software and sign-in

Git and Github Activity – Getting Started

Fork the Repository

- Go to <https://github.com/t2laird/course-toolkit-sp26>
- Click the "Fork" button in the top right corner
- This creates your own copy of the repository under your GitHub account
- You'll see it at: github.com/your-username/course-toolkit-sp26

Git and Github Activity – Getting Started

Clone Your Fork

- Open Github Desktop
- Click on Current Repository
- Click Add
- Click Clone Repository
- Select the Course Toolkit Repository from the list that comes up

Git and Github Activity – Getting Started

Create a Branch

Branches let you work on changes without affecting the main code.

- In Github Desktop Click on Current Branch
- Click New Branch
- Give it a name based on what you are doing
 - Alternative naming examples:
 - add-resource-john
 - Feature/email-validator
 - add-pandas-guide
- Publish your branch
- Verify you're on your new branch
 - Check what branch it has listed in the Current Branch of Github Desktop



Git and Github Activity – Your First Contribution

Add yourself as a contributor

- Open the README.md file (it's in the main course-toolkit folder)
- Find the "Contributors" section
- Add your name to the list in this format
 - First Name Last Name
- Save the README.md file

Git and Github Activity – Your First Contribution

Add a Resource

- Create a markdown file in the resources/ folder
- Name it something descriptive: resources/pandas-cheatsheet.md
- Check the Issues tab for ideas or contribute your own
- See the README File for more instructions on templating

Add a Code Snippet

- Create a code file in the code-snippets/ folder
- Name it descriptively: code-snippets/language_email_validator.[extension]
- Keep it small and useful (10-20 lines)
- See the README File for more instructions



Git and Github Activity – Your First Contribution

Create Your File

- Open the course-toolkit folder in your text editor or IDE
- Navigate to resources/ or code-snippets/
- Create your new file
- Add your content
- Save the file

Git and Github Activity – Your First Contribution

Stage and Commit Your Changes

- Check what changed
 - You should see a list in the left hand side of Github Desktop with your new files and any documents you have changed within the Repository
- Add a Summary of what is changing
 - This should be short and just a few words
- Add a Description of what is changing
 - This is a more detailed description of the updates that you are making
- Click Commit
 - This will commit the changes to your branch
- Click Push Origin in Github Desktop
 - This will push the changes you made to the cloud



Git and Github Activity – Pull Request Process

Create a Pull Request

- On Github Desktop you should now see a Preview Pull Request Option
 - Click it!
- This will open Github online

Git and Github Activity – Pull Request Process

Write a Good Pull Request Description

Your PR should include:

- Title: Clear and concise
 - Good: "Add pandas cheat sheet resource"
 - Bad: "Update"
- Description: Answer these questions:
 - What did you add?
 - Why is it useful?
 - Does it close an issue? (If yes, write: "Closes #3")
- Example:

Added a resource about pandas cheat sheet from the official documentation. This is helpful for quick reference when working with data frames.

Closes #2

Git and Github Activity – Pull Request Process

Submit Your PR

- Review your changes one more time
- Click "Create pull request"
- You're done! Your PR is now visible to everyone
- Check the "Pull requests" tab on the main repository to see yours

Git and Github Activity – Code Review

What is Code Review?

Code review is when team members look at each other's code before it gets merged.

- Benefits:
 - Catch mistakes or bugs
 - Share knowledge
 - Improve code quality
 - Learn from each other

In professional settings, code review is standard practice!

Git and Github Activity – Code Review

Completing Code Review

- Look at the "Pull requests" tab on the main repository.
- Review 1-2 PRs from your classmates
 - Click on a PR that isn't yours
 - Read what they added
 - Leave constructive feedback

Git and Github Activity – Code Review

How to Review

- Click on "Files changed" tab in the PR
- You'll see what they added (green lines) or removed (red lines)
- Click the "+" button next to any line to leave a comment
- Or click "Review changes" button at the top to leave general feedback

What to Look For

- Good comments include:
 - "This looks helpful! I didn't know about this tool."
 - "Could you add an example of when to use this?"
 - "Small typo in line 5: 'recieve' should be 'receive'"
 - "Nice work! This is very clear."
- Avoid:
 - "This is bad"
 - Leaving no comments at all
 - Being overly critical without being constructive

Git and Github Activity – Code Review

Approve or Request Changes

- After reviewing, click "Review changes" and choose:
 - Approve: Looks good, ready to merge!
 - Comment: Leaving feedback but not blocking
 - Request changes: Something needs to be fixed before merging

For today, use "**Approve**" or "**Comment**" unless you spot a real issue.

Git and Github Activity – Merge Conflicts

What's a Merge Conflict?

A merge conflict happens when:

- Two people edit the same part of the same file
- Git doesn't know which version to keep
- This is normal and happens all the time!

Git and Github Activity – Merge Conflicts

When you see a conflict in a file, it looks like this:

```
<<<<<< HEAD
```

This is the current version in main

```
=====
```

This is your version in your branch

```
>>>>>> your-branch-name
```

Git and Github Activity – Merge Conflicts

Resolving Conflicts Steps:

- Open the file with conflicts
- Find the conflict markers (<<<<<<, =====, >>>>>>)
- Decide what to keep:
 - Keep one version, or
 - Keep both versions, or
 - Combine them in a new way
- Delete ALL conflict markers
- Save the file

Git and Github Activity – Merge Conflicts

Don't Panic!

Remember:

- Conflicts are normal in collaborative work
- They don't mean you did something wrong
- Take your time to understand what each person changed
- When in doubt, ask!
- GitHub also has a web interface for simple conflicts

Git and Github Activity – Merge Conflicts

Let's Solve One Together

We're going to intentionally look at a conflict and resolve it as a class.

- We'll walk through:
 - Identifying the conflict
 - Understanding what each person changed
 - Deciding what to keep
 - Resolving it